

Frequency Counter Design and Verification Report

Verilog RTL Design • Vivado Behavioral Simulation

Digital VLSI / RTL Student Project

SUBMITTED BY

KISHORE KUMAR C

ELECTRONICS ENGINEERING(VLSI DESIGN AND TECHNOLOGY)

KARPAGAM COLLEGE OF ENGINEERING

REPOSITORY: <https://github.com/kishorekumar3728-gif/Digital-Frequency-Counter.git>

Project Title	Design and Simulation of a Parameterized Digital Frequency Counter Using Verilog HDL
HDL	Verilog
Simulation Platform	Xilinx Vivado Behavioral Simulation
Reference Design	Matt Venn – frequency_counter repository
Verification	Clocked testbench with periodic 10 MHz input signal

Contents

1. Abstract
2. Objectives
3. Frequency Counter Fundamentals
4. System Architecture
5. RTL Design
6. Verification Testbench
7. Simulation and Waveform Analysis
8. Results
9. Design Considerations and Limitations
10. Conclusion
11. References

1. Abstract

A digital frequency counter measures the frequency of an input digital signal by counting its rising edges during a defined measurement interval. This project implements a parameterized frequency counter using Verilog HDL. The input signal is synchronized to the reference clock, rising edges are detected, and an edge counter accumulates the number of detected transitions during a measurement window. At the end of the window, the measured count is transferred to a frequency result register and a one-clock measurement-done pulse is generated. The design was verified using Xilinx Vivado behavioral simulation.

2. Objectives

- Design a synthesizable RTL frequency counter using Verilog HDL.
- Synchronize an incoming digital signal before processing it.
- Detect rising edges of the synchronized input signal.
- Count input edges over a programmable reference-clock window.
- Register the measured result and generate a measurement-done indication.
- Verify the RTL using a self-contained simulation testbench and waveform analysis.

3. Frequency Counter Fundamentals

The measurement principle is based on counting the number of input cycles that occur during a known measurement interval:

$$N = f_{in} \times T_{measure}$$

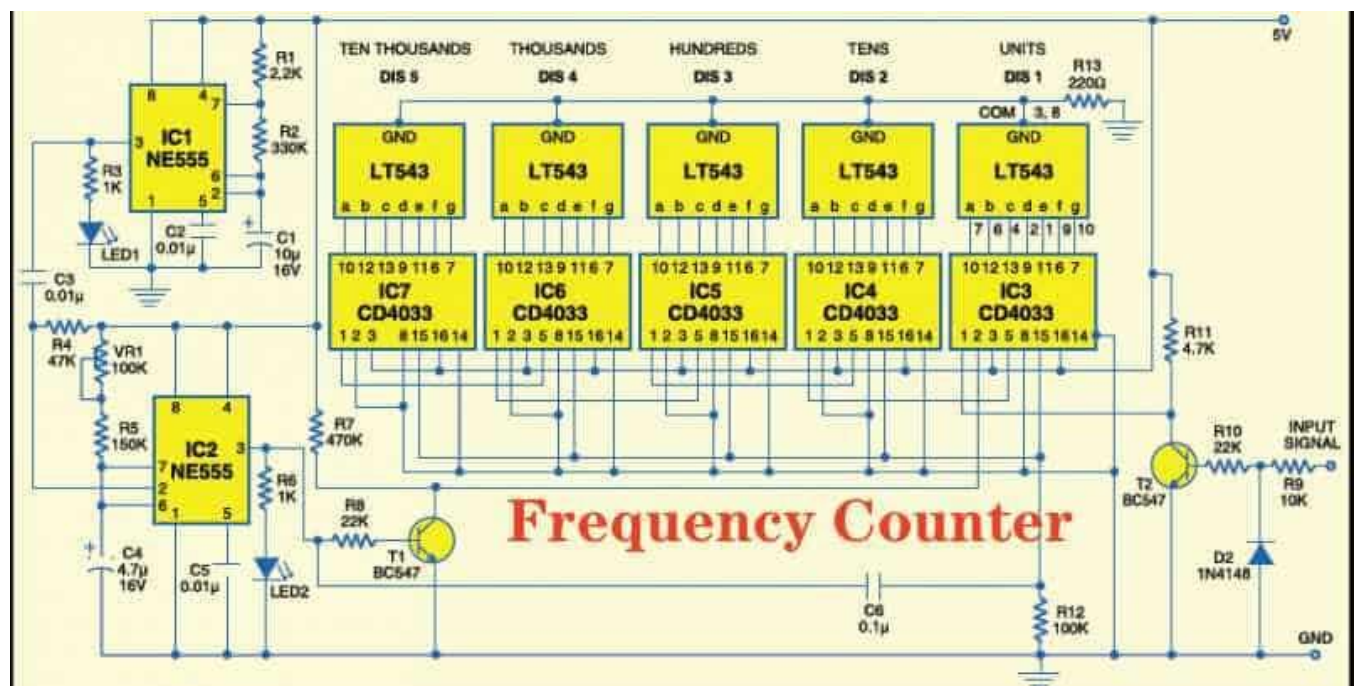
where N is the number of detected rising edges, f_{in} is the input frequency, and $T_{measure}$ is the measurement-window duration. Therefore, when the measurement interval is known, the counted edges provide the frequency information.

3.1 Design Parameters Used in Simulation

Parameter	Value	Purpose
Reference clock	100 MHz	Main synchronous clock
Clock period	10 ns	Reference-clock period
Measurement window	1000 clock cycles	Defines measurement interval
Measurement time	10 μ s	1000×10 ns
Input signal	10 MHz	Test frequency
Expected edge count	100	$10 \text{ MHz} \times 10 \mu\text{s}$

4. System Architecture

The design consists of an input synchronizer/edge detector and a measurement engine. The measurement engine contains a reference-clock window counter and an input edge counter. When the window expires, the measured edge count is stored in the frequency_count register.



4.1 Signal Flow

- signal_in is the digital input whose frequency is being measured.
- A two-stage synchronizer transfers the input into the reference-clock domain.
- A previous-sample register is used to generate a one-clock rising_edge pulse.
- edge_count increments whenever rising_edge is asserted.
- window_counter counts reference-clock cycles until the programmed measurement window ends.
- At the window boundary, frequency_count is updated and measurement_done pulses for one clock.

5. RTL Design

5.1 Edge Detector RTL

```
`timescale 1ns/1ps

module edge_detector (
    input  wire clk,
    input  wire reset,
    input  wire signal_in,
    output wire rising_edge
);

    reg sync_ff1;
    reg sync_ff2;
    reg signal_prev;

    always @(posedge clk) begin
        if (reset) begin
            sync_ff1    <= 1'b0;
            sync_ff2    <= 1'b0;
            signal_prev <= 1'b0;
        end
        else begin
            sync_ff1    <= signal_in;
            sync_ff2    <= sync_ff1;
            signal_prev <= sync_ff2;
        end
    end

    assign rising_edge = sync_ff2 & ~signal_prev;

endmodule
```

5.2 Frequency Counter RTL

```
`timescale 1ns/1ps

module frequency_counter #(
    parameter WINDOW_CYCLES = 1000
) (
    input  wire      clk,
    input  wire      reset,
    input  wire      signal_in,

    output reg [31:0] frequency_count,
    output reg [31:0] edge_count,
    output reg        measurement_done
);

    wire rising_edge;
    reg [31:0] window_counter;

    edge_detector edge_detector_inst (
        .clk          (clk),
        .reset        (reset),
        .signal_in    (signal_in),
        .rising_edge  (rising_edge)
    );
```

```

);

always @(posedge clk) begin
    if (reset) begin
        window_counter    <= 32'd0;
        edge_count        <= 32'd0;
        frequency_count    <= 32'd0;
        measurement_done   <= 1'b0;
    end
    else begin
        measurement_done   <= 1'b0;

        if (window_counter == WINDOW_CYCLES - 1) begin
            if (rising_edge)
                frequency_count <= edge_count + 1'b1;
            else
                frequency_count <= edge_count;

            measurement_done <= 1'b1;
            window_counter   <= 32'd0;
            edge_count       <= 32'd0;
        end
        else begin
            window_counter <= window_counter + 1'b1;

            if (rising_edge)
                edge_count <= edge_count + 1'b1;
        end
    end
end

endmodule

```

6. Verification Testbench

The testbench generates a 100 MHz reference clock and a 10 MHz input signal. Reset is asserted initially and then released. The simulation is run long enough to observe the completion of the 10 μ s measurement window.

```

`timescale 1ns/1ps

module frequency_counter_tb;

    reg clk;
    reg reset;
    reg signal_in;

    wire [31:0] frequency_count;
    wire [31:0] edge_count;
    wire measurement_done;

    frequency_counter #(
        .WINDOW_CYCLES(1000)
    ) dut (
        .clk            (clk),
        .reset          (reset),
        .signal_in      (signal_in),

```

```

        .frequency_count (frequency_count),
        .edge_count      (edge_count),
        .measurement_done(measurement_done)
    );

    // 100 MHz reference clock: 10 ns period
    always #5 clk = ~clk;

    // 10 MHz input: 100 ns period
    always #50 signal_in = ~signal_in;

    initial begin
        clk      = 1'b0;
        reset    = 1'b1;
        signal_in = 1'b0;

        #100;
        reset = 1'b0;

        #15000;

        $finish;
    end

endmodule

```

6.1 Expected Verification

The reference clock has a 10 ns period. With WINDOW_CYCLES = 1000, the measurement interval is 10,000 ns or 10 μ s. The input signal has a 100 ns period, corresponding to 10 MHz. Therefore the expected number of input rising edges in the measurement interval is approximately 100.

7. Simulation and Waveform Analysis

The RTL was simulated in Vivado using behavioral simulation. The final simulation was run for 15 μ s so that the complete first measurement window and part of the following measurement could be observed.

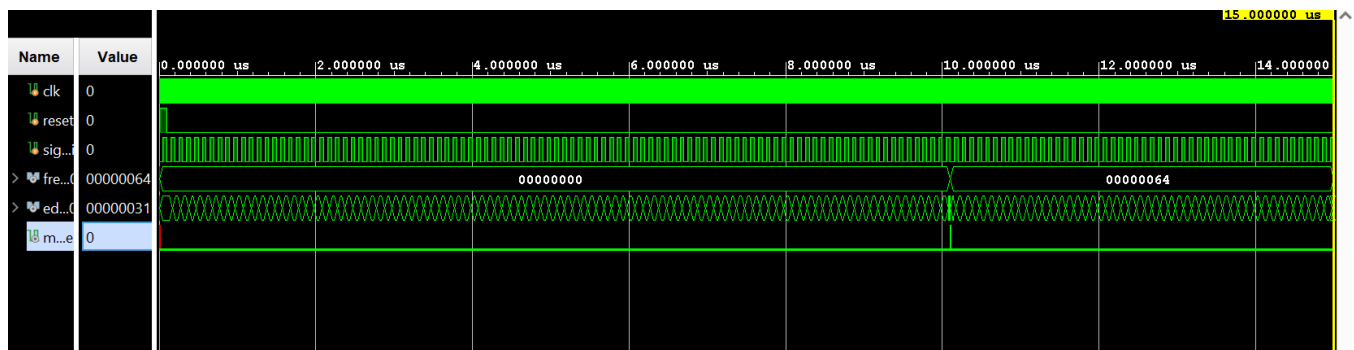


Figure 1. Vivado behavioral simulation waveform for the frequency counter.

7.1 Waveform Interpretation

- The reference clock toggles continuously with a 10 ns period.
- Reset is initially asserted and is released at approximately 100 ns.
- The input signal toggles periodically with an approximately 100 ns period.

- The synchronized edge detector produces rising-edge events corresponding to input transitions.
- `edge_count` increases as input rising edges are detected.
- At approximately 10 μ s, the measurement window completes.
- `frequency_count` changes from 0 to 0x00000064, which is decimal 100.
- `measurement_done` produces a short pulse indicating that a new measurement is available.

8. Results

Measured Item	Observed Result	Status
Reference clock	100 MHz	PASS
Input signal	10 MHz	PASS
Measurement window	10 μ s	PASS
Frequency result	0x64 = 100 edges	PASS
Measurement-done indication	One-clock pulse	PASS
Edge counting	Continuous incrementing	PASS

The simulation demonstrates that the design detects the periodic input signal, counts its rising edges, and updates the frequency result after the programmed measurement interval. The observed `frequency_count` value of 0x64 corresponds to decimal 100, matching the expected number of input cycles during a 10 μ s window for a 10 MHz input.

9. Design Considerations and Limitations

- The input signal is synchronized to the reference clock before edge detection.
- The measurement resolution depends on the selected measurement-window duration.
- The maximum measurable frequency is limited by the sampling/reference-clock relationship and implementation timing.
- The current result register reports the counted number of edges per measurement window; converting this count to an absolute value in Hz requires the known measurement-window duration.
- The current project focuses on the core RTL measurement engine and does not include FPGA board-specific seven-segment display hardware.

10. Conclusion

A parameterized digital frequency counter was designed using Verilog HDL and verified through Vivado behavioral simulation. The design uses input synchronization, rising-edge detection, a reference-clock measurement window, an edge counter, and a result register. For a 10 MHz input and a 10 μ s measurement interval, the simulation produced a measured edge count of 100, confirming the intended operation of the RTL design.

11. References

1. Matt Venn, frequency_counter repository, source modules including frequency_counter.v, edge_detect.v, and seven_segment.v.
2. Xilinx Vivado Design Suite – RTL simulation environment used for behavioral verification.

3. VLSI Student Projects document supplied for the project selection and scope: Frequency Counter – Measure frequency using digital counters.